

few of these modules with bugs can cause problems with the system, and need to be debugged.

As these modules are inserted in the kernel at a later stage, GDB has no knowledge of the relevant symbol information, or the location of the module in memory. We need to feed this information to GDB, once the module is loaded, in order to debug the module.

GDB has a command, *add-symbol-file*, which takes the '.ko' module file (which you are trying to debug) as its first argument, and the address of the '.text' section of the module as the second argument. The '.text' address can be obtained from `/sys/module/<module name>/sections/.text`.

Let's consider an example, using the module *loop.ko*. In the UML instance:

1. Insert the module *loop.ko* in the UML kernel. (If it is not compiled, you can recompile *loop.ko* and copy it to the UML system.)

```
# insmod/modprobe loop.ko
```

2. Obtain the address from `/sys/module/loop/sections/.text` (see Figure 9):

```
# cat /sys/module/loop/sections/.text
```

3. To debug the *loop.ko* module, we need to prepare a sample image file and format it, ready to be mounted at a later stage:

```
# dd if=/dev/zero of=fs.img count=2 bs=1024k
# mkfs.ext3 fs.img
```

On the host system:

1. In a different terminal window (which you started the UML instance from), find the process ID of the UML instance.
2. Attach GDB to the running UML instance, specifying the PID. For example:

```
$ gdb uml-linux-image 8892
```

3. In GDB, load the debug symbol information for the module:

```
add-symbol-file /code/kernel/lfy/linux-kernel/drivers/block/loop.ko 0x7187c000
```

(The last argument here is the *.text* address of the loop module, obtained in the second step we ran in the UML instance, above.)

4. Test whether the module is properly loaded:

```
p loop_unplug
```

(*loop_unplug* is a function in the *drivers/block/loop.c* file. This GDB command should show the *.text* address we used earlier. Once you see the *.text* address, it implies you are able to access the module through GDB.)

5. Now, put a breakpoint on the *loop_unplug()* function:

```
# b loop_unplug
```

6. Finally, type '*c*' at the *gdb* prompt, to continue running until the breakpoint is encountered. Figure 10 illustrates the above steps.

Back in the UML instance, we can activate our breakpoint with the following steps:

```
# mkdir test
# mount -o loop fs.img test
```

This should hit the breakpoint in the running GDB instance in the host system. We can then view and debug the code of the module.

The purpose of this article was to provide an introduction to UML, and a step-by-step guide to setting up a UML system and debugging the kernel and modules. The methods mentioned in this article are one of the many available to set up and play around with UML. For more knowledge on the topic, you can subscribe to the UML mailing lists at <https://lists.sourceforge.net/lists/listinfo/user-mode-linux-devel> or visit the UML home page at <http://user-mode-linux.sourceforge.net>. 

By: **Surya Prabhakar**

The author is an engineering advisor in the Product Group at Dell India R&D centre, Bengaluru and has 8 years of experience in Linux. He spends most of his time hacking and playing with Linux. He can be reached at surya_prabhakar@dell.com

THE COMPLETE MAGAZINE ON OPEN SOURCE

LINUX

BETA ForYou

Your favourite Linux Magazine is
now on the Web, too.

LinuxForU.com

Follow us on Twitter @LinuxForYou